EXP 801 DS Wheat Heat Drought

— Ed Chaplin (USyd), *Apr–June 2025* —

PREDICTION PLOTS

```
#"### Load libraries
library(knitr)
#knitr::spin("Exp801.anat.pred.v1.r")
library(ggplot2)
library(dae)
library(asremlPlus)
library(tidyverse)
library(patchwork)
source("utilities.R")
```

Set up constants and files

```
options(width=90)
opts_chunk$set("tidy" = FALSE, "dev" = "png", "dpi" = 300, fig.path = "figures/",
               "fig.width" = 7, "fig.height" = 8.5)

load("../data/anat.Analyses.v2.RData")
saveFile.pred <- "../data/anat.Preds.v1.Rdata"
```

Generate and export full model predictions to inform correlations for Ed

```
Diffs.full <- mapply(function(mod) {
  mod$preds
}, models, SIMPLIFY = FALSE)
Diffs.full <- do.call(rbind, Diffs.full)
Diffs.full$trait <- row.names(Diffs.full)
Diffs.full <- Diffs.full %>% separate(trait, into = c("Trait", "rmv")) %>%
  select(-rmv)

##export these predictions
save(Diffs.full, file = "../data/anat.diffs.full.rda")
```

```
respnames.anat <- c("Stomatal Count", "Stomatal Density", "Stomatal Area",
                    "Guard Cell Width", "Guard Cell Length", "Pore Area", "italic(g)[smax[a]]")
respunits.anat <- c("", "mm^-2", "mu*mm^-2", "mu*m", "mu*m", "", "mol~m^-2~s^-1")
respnames <- respnames.anat
respunits <- respunits.anat

Diffs.chos <- mapply(function(mod)
{
  terms <- mod$chosen.terms; if (is.null(terms)) terms <- "1"
  mod.chosen <- paste0("~ ",paste(terms, collapse = " + "))
  diffs <- linTransform(mod$diffs,
               linear.transformation = as.formula(mod.chosen),
               wald.tab = mod$wald.tab,
               error.intervals="half",
```

```
              LSDtype = "factor", LSDby = c("Rm_Temperature", "Watering"),
              avsed.tolerance = NA,
              tables = "none", Vmatrix = TRUE)
  minLSD <- findLSDminerrors(diffs, LSDtype = "fact", LSDby =c("Rm_Temperature",
                                                               "Watering"))
  diff <- redoErrorIntervals(diffs, error.intervals = "half",
                             LSDtype = "supplied", LSDby = c("Rm_Temperature",
                                                             "Watering"),
                             LSDsupplied = minLSD["LSD"],
                             avsed.tolerance = NA)
}, models, SIMPLIFY = FALSE)
```

Returning to regular programming to produce plots, etc.

```
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : Some factors in the LSDby are not in the linear.transformation
## submodel
```

```
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
```

```
## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`
```

```
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
```

```
## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`
```

```
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : Some factors in the LSDby are not in the linear.transformation
## submodel
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
```

```
## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`
```

```
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
```

```
## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`
```

```
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : Some factors in the LSDby are not in the linear.transformation
## submodel
```

```
## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`
## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`
```

```
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : Some factors in the LSDby are not in the linear.transformation
## submodel
```

```
## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`
## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`
```

```
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : Some factors in the LSDby are not in the linear.transformation
## submodel
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
```

```
## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`
```

```
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
```

```
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error

## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`

## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : Some factors in the LSDby are not in the linear.transformation
## submodel
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error

## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`

## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error

## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`

## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : Some factors in the LSDby are not in the linear.transformation
## submodel
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in linTransform.alldiffs(mod$diffs, linear.transformation =
## as.formula(mod.chosen), : LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error

## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`

## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
```

```
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
## Warning in FUN(X[[i]], ...): LSD calculated for a single prediction- applies to two
## independent predictions with the same standard error
```

```
## Joining with `by = join_by(fac.comb)`
## Joining with `by = join_by(Rm_Temperature, Watering)`
```

Calculate the errors from using the selected LSDs

```
(do.call(rbind, lapply(Diffs.chos, function(d) d$LSD[c("c","assignedLSD", "falsePos", "falseNeg")])))
```

```
##                                      c  assignedLSD falsePos falseNeg
## StomatalCount.7,Heat,Limited        45   3.20041392        0        0
## StomatalCount.7,Heat,Unlimited      45   3.20041392        0        0
## StomatalCount.8,Ambient,Limited      0   1.47944666       NA       NA
## StomatalCount.8,Ambient,Unlimited    0   1.47944666       NA       NA
## StomatalDensity.7,Heat,Limited      45   6.22248083        0        0
## StomatalDensity.7,Heat,Unlimited    45   6.22248083        0        0
## StomatalDensity.8,Ambient,Limited    0   2.87644934       NA       NA
## StomatalDensity.8,Ambient,Unlimited  0   2.87644934       NA       NA
## StomatalArea.7,Heat,Limited         45 177.37156673        0        0
## StomatalArea.7,Heat,Unlimited       45 177.37156673        0        0
## StomatalArea.8,Ambient,Limited      45 172.34832741        0        0
## StomatalArea.8,Ambient,Unlimited    45 172.34832741        0        0
## StomatalWidth.7,Heat,Limited        45   2.95691351        0        0
## StomatalWidth.7,Heat,Unlimited      45   2.95691351        0        0
## StomatalWidth.8,Ambient,Limited     45   4.17941311        0        0
## StomatalWidth.8,Ambient,Unlimited   45   4.19930036        0        0
## StomatalLength.7,Heat,Limited       45   4.32200151        0        0
## StomatalLength.7,Heat,Unlimited     45   4.32200151        0        0
## StomatalLength.8,Ambient,Limited     0   1.77834900       NA       NA
## StomatalLength.8,Ambient,Unlimited   0   1.77834900       NA       NA
## PoreArea.7,Heat,Limited             45 125.17551268        0        1
## PoreArea.7,Heat,Unlimited           45 125.17551268        0        1
## PoreArea.8,Ambient,Limited           0  49.65447289       NA       NA
## PoreArea.8,Ambient,Unlimited         0  49.65447289       NA       NA
## gsmax.7,Heat,Limited                45   0.18162850        0        0
## gsmax.7,Heat,Unlimited              45   0.18162850        0        0
## gsmax.8,Ambient,Limited              0   0.06994288       NA       NA
## gsmax.8,Ambient,Unlimited            0   0.06994288       NA       NA
```

Extract preds

```
Preds.resp <- lapply(Diffs.chos, function(diffs) diffs$predictions)

lower.col <- "lower.halfLeastSignificant.limit"
upper.col <- "upper.halfLeastSignificant.limit"
```

Plot a single trait

```
plotResponse <- function(k=1, response=NULL, dy1=NULL, dy2=dy1/2,
                         keep.xaxis = TRUE, plot.title = NULL)
{
  if (is.null(response))  response <- responses[k]  else  k <- match(response,responses.anat)

  tmp <- Preds.resp[[response]]

  tmp <- within(tmp,
                {
                  Line <- as.character(Line)
                  Line <- factor(Line)
                  Rm_Temperature <- fac.recast(Rm_Temperature,
                                               newlevels = c("CER7_Heat", "CER8_Ambient"))
                })
  vals <- tmp[c(upper.col, lower.col, "predicted.value")] # allows for null error-bars
  ymin <- min(tmp[[lower.col]])
  yname <- respnames[k]
  yunit <- respunits[k]
  ytitl <- yname; if (yunit!="") ytitl <- paste0(ytitl," (",yunit,")")
  ytitl <- str2expression(gsub(" ", "~", ytitl))

  fix.dy <- !is.null(dy1)

  plt <- ggplot(tmp, aes(x=Line, y=predicted.value, fill = Line)) +
    facet_grid(rows = vars(Rm_Temperature), cols = vars(Watering)) +
    geom_bar(stat="identity", colour = "black", width = 0.75,
             position = position_dodge(width = 0.95), fill = "grey50") +
    #scale_fill_grey(start = 0.2, end = 0.8) +
    #geom_point(size=1.5, position = position_dodge(width = 0.95)) +
    # scale_shape_manual(values = c(4,NA), drop=FALSE, guide = "none") +
    #scale_shape_manual(values = c(yes = 4, no = NA), guide = "none") +
    geom_errorbar(aes(ymin=lower.halfLeastSignificant.limit,
                      ymax=upper.halfLeastSignificant.limit),
                  width=0.5, alpha=0.5, position = position_dodge(width = 0.95)) +
    (if (fix.dy & ymin >= 0) data.yscale(vals, dy1, dy2, ymin=0)) +
    (if (fix.dy & ymin <  0) data.yscale(vals, dy1, dy2)) +
    labs(y = ytitl,
         x = NULL) +
    theme_bw() +
    if (keep.xaxis){
    theme(legend.position = "none",
                       axis.text.x = element_text(size=9, hjust=1, vjust=0.5, angle=90),
                       strip.text = element_text(size=11, face="bold"))}
  else{
    theme(legend.position = "none",
          strip.text = element_text(size=11, face="bold"),
          axis.title.x = element_blank(),
          axis.text.x  = element_blank(),
          axis.ticks.x = element_blank())
  }
  plt <- plt +
    if(!is.null(plot.title)){
```

```
      ggtitle(plot.title)
    }
  plt
}


cap.trait <- "Estimated marginal means (EMMs) for xxxx for
two Room_Temperature combinations, two Watering treatments and ten Genotypes.
Error bars are an EMM $\\pm$ half-LSD (5%).
Any two EMMs for the same Room_Temperature and Watering combination
are significantly different if their error bars do not overlap."
```

```
plotResponse(response = "StomatalDensity", dy1=20)
```

```
SD <- plotResponse(response = "StomatalDensity", dy1=20, keep.xaxis = FALSE,
                   plot.title = "(a)")
```

```
plotResponse(response = "StomatalArea", dy1=500)
```

```
SA <- plotResponse(response = "StomatalArea", dy1=500, keep.xaxis = FALSE,
                   plot.title = "(b)")
```

```
plotResponse(response = "StomatalWidth", dy1=10)
```

```
GCW <- plotResponse(response = "StomatalWidth", dy1=10, keep.xaxis = FALSE,
                    plot.title = "(c)")
```

```
plotResponse(response = "StomatalLength", dy1=20)
```

```
GCL <- plotResponse(response = "StomatalLength", dy1=20, keep.xaxis = FALSE,
                    plot.title = "(d)")
```

```
plotResponse(response = "gsmax", dy1=0.5)
```

```
Gsmax <- plotResponse(response = "gsmax", dy1=0.5, plot.title = "(e)")

##put together a combined plot of these figures
SD / SA / GCW / GCL / Gsmax
```

```
ggsave("figures/AnatomicalTraitPredictions.png", width = 10, height = 18)
ggsave("figures/AnatomicalTraitPredictions.pdf", width = 10, height = 18)
ggsave("figures/AnatomicalTraitPredictions.tiff", width = 10, height = 18)
```

Compress file to save memory

```
save.image(saveFile.pred, compress = "xz")
```

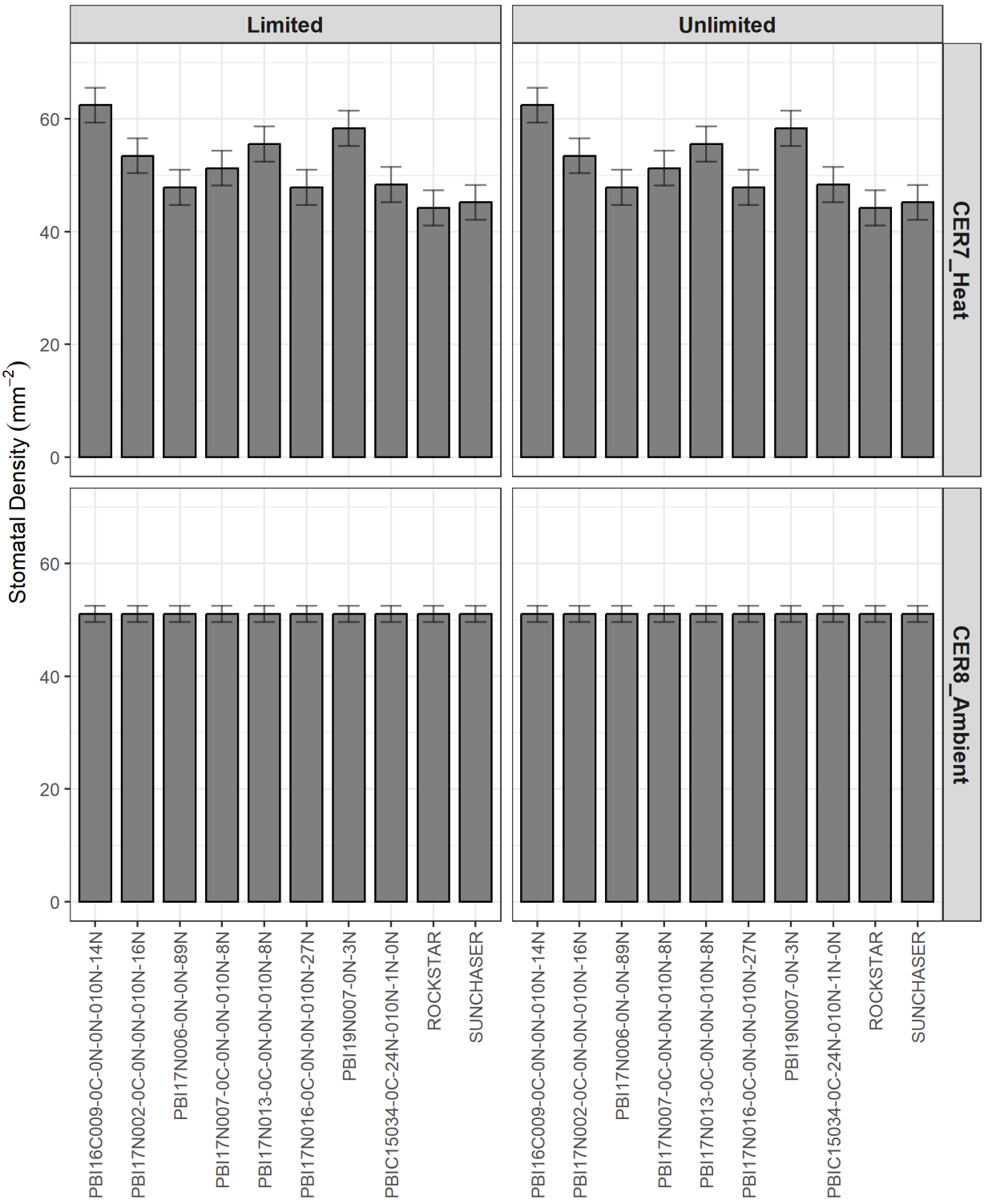


Figure 1: Estimated marginal means (EMMs) for Stomatal Density for two Room_Temperature combinations, two Watering treatments and ten Genotypes. Error bars are an EMM ± half-LSD (5%). Any two EMMs for the same Room_Temperature and Watering combination are significantly different if their error bars do not overlap.

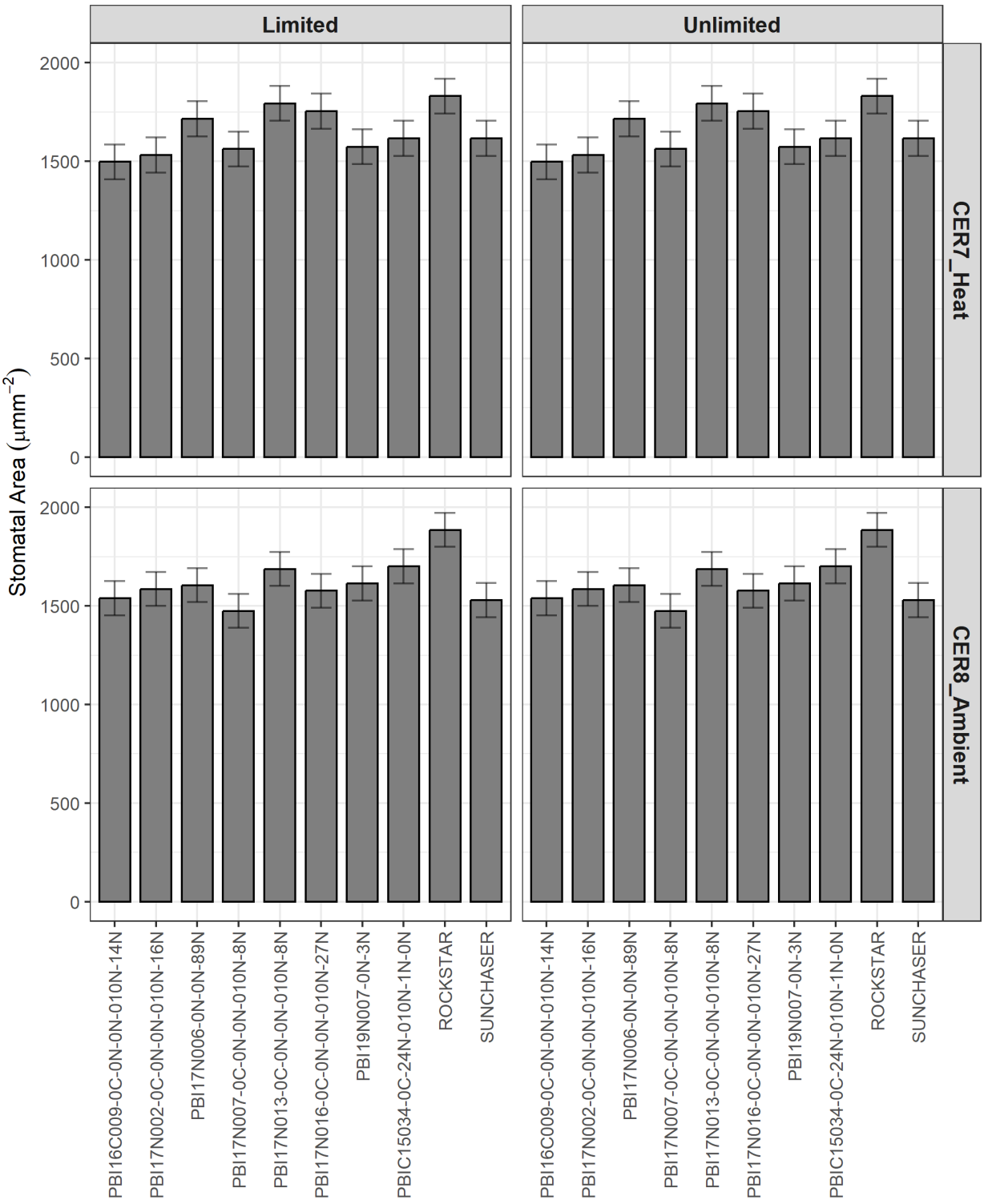


Figure 2: Estimated marginal means (EMMs) for Stomatal Area for two Room_Temperature combinations, two Watering treatments and ten Genotypes. Error bars are an EMM ± half-LSD (5%). Any two EMMs for the same Room_Temperature and Watering combination are significantly different if their error bars do not overlap.

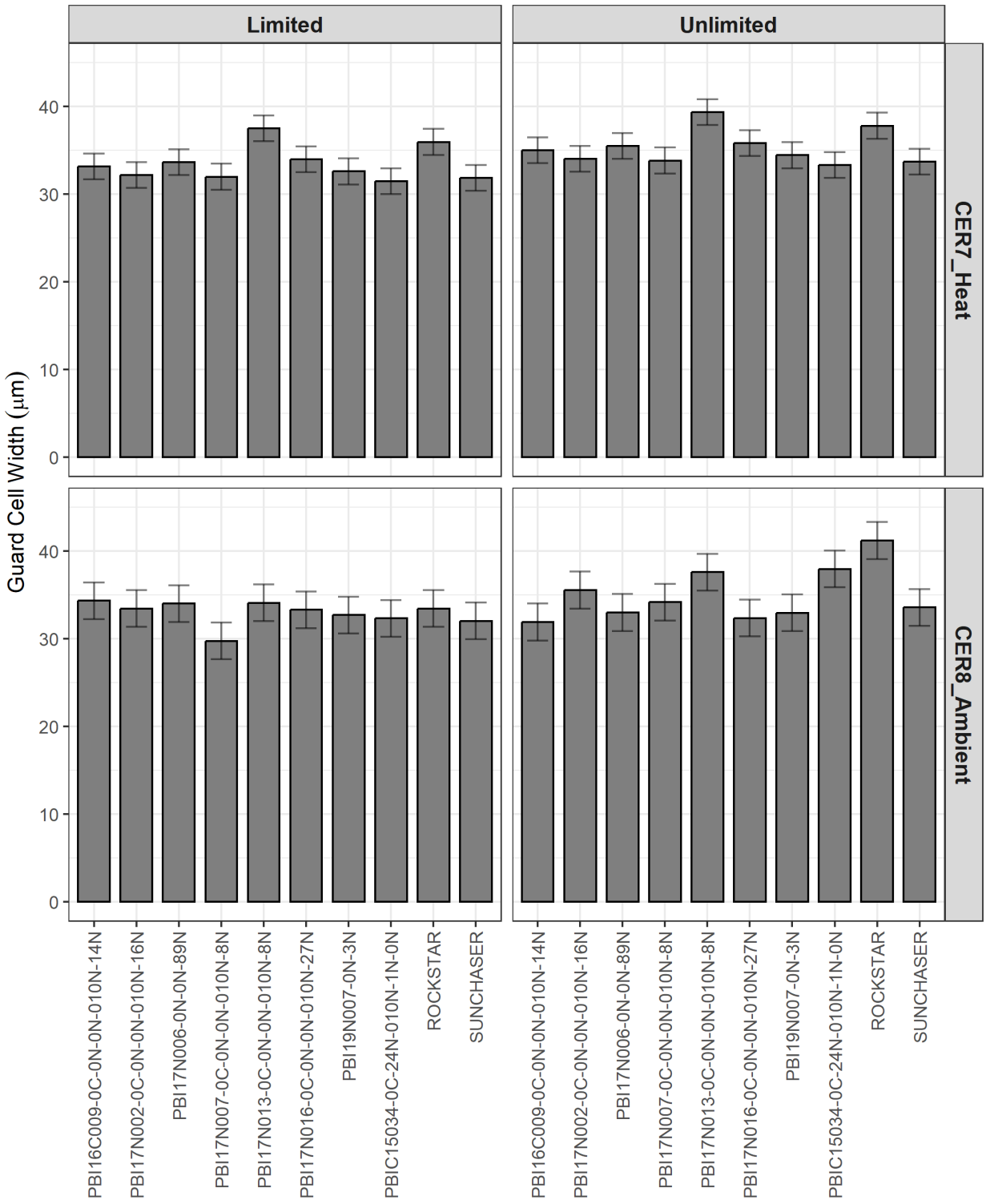


Figure 3: Estimated marginal means (EMMs) for Guard Cell Width for two Room_Temperature combinations, two Watering treatments and ten Genotypes. Error bars are an EMM ± half-LSD (5%). Any two EMMs for the same Room_Temperature and Watering combination are significantly different if their error bars do not overlap.

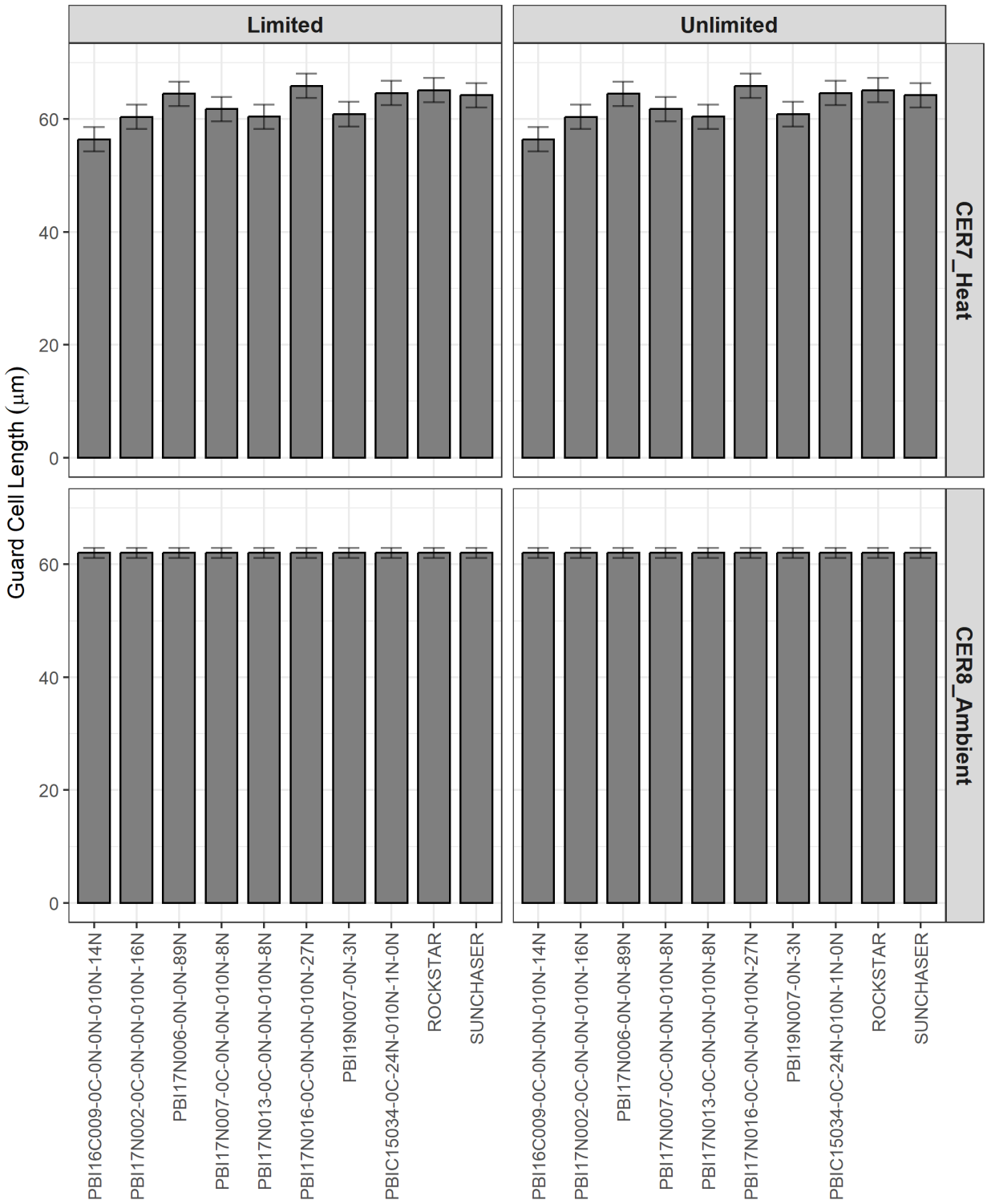


Figure 4: Estimated marginal means (EMMs) for Stomatal Cell Length for two Room_Temperature combinations, two Watering treatments and ten Genotypes. Error bars are an EMM ± half-LSD (5%). Any two EMMs for the same Room_Temperature and Watering combination are significantly different if their error bars do not overlap.

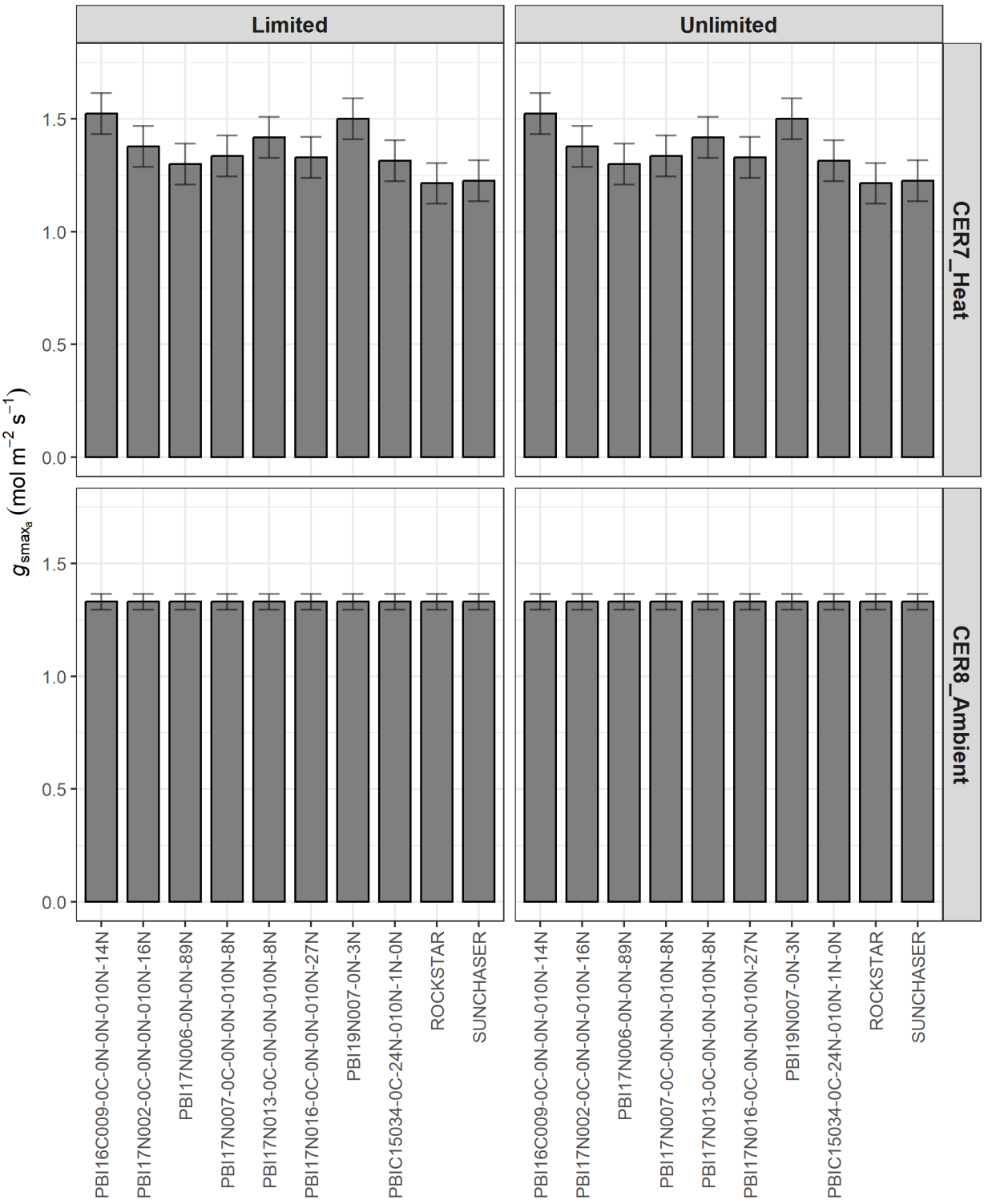


Figure 5: Estimated marginal means (EMMs) for Gsmax_a for two Room_Temperature combinations, two Watering treatments and ten Genotypes. Error bars are an EMM ± half-LSD (5%). Any two EMMs for the same Room_Temperature and Watering combination are significantly different if their error bars do not overlap.

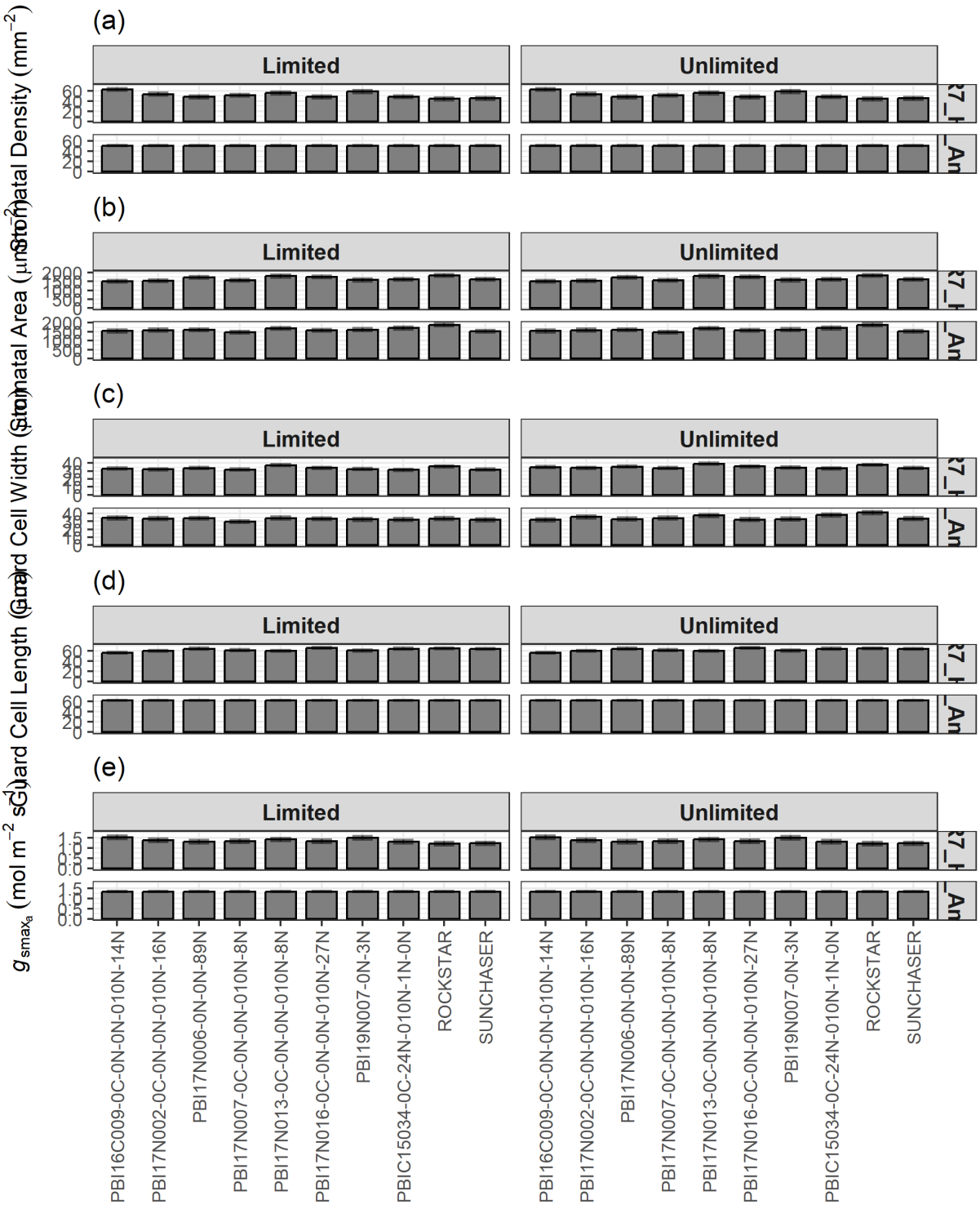


Figure 6: Estimated marginal means (EMMs) for Gsmax_a for two Room_Temperature combinations, two Watering treatments and ten Genotypes. Error bars are an EMM ± half-LSD (5%). Any two EMMs for the same Room_Temperature and Watering combination are significantly different if their error bars do not overlap.